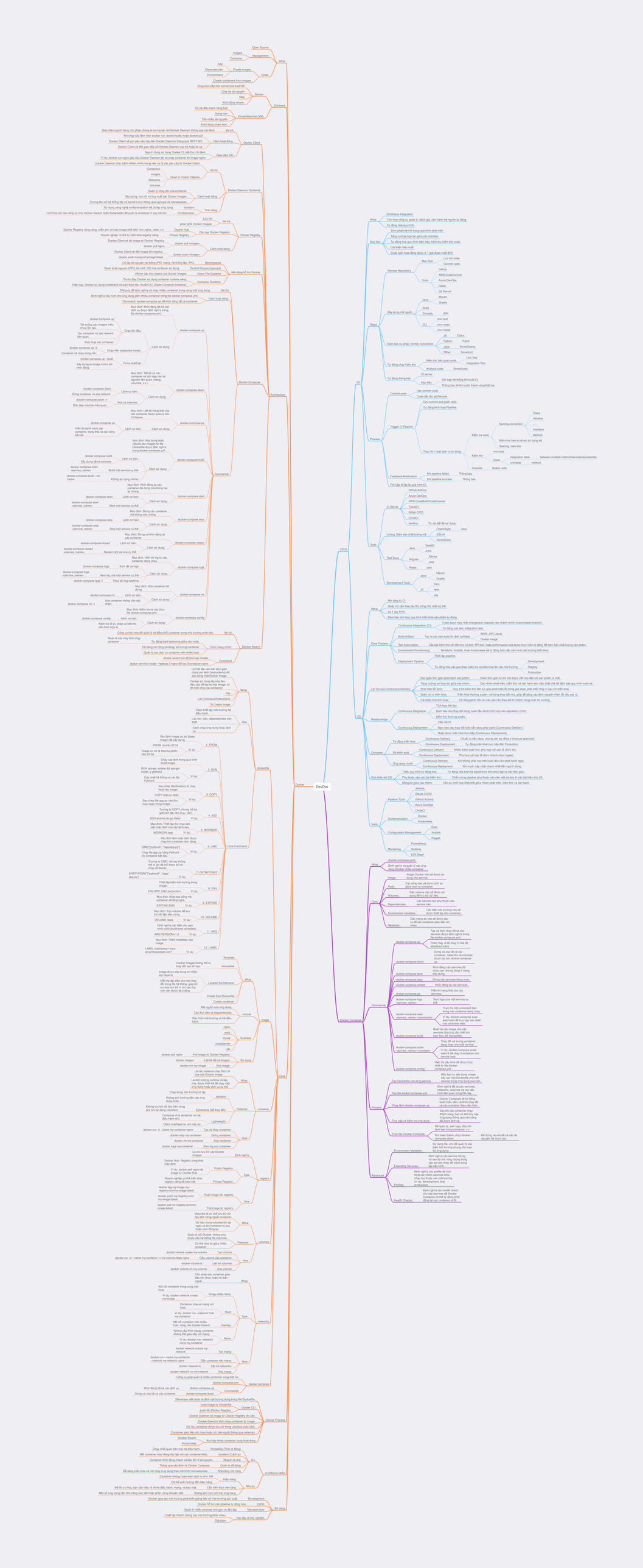

DevOps

- Docker
  - What
    - Open Source
    - Management
      - Images
      - Container
    - Goals
      - Create images
        - App
        - Dependencies
        - Environment
      - Create containers from images
  - Compare
    - Docker
      - Chạy trực tiếp trên kernel của host OS
      - Chia sẻ tài nguyên
      - Nhẹ
      - Khởi động nhanh.
    - Virtual Machine (VM)
      - Có hệ điều hành riêng biệt
      - Nặng hơn
      - Tốn nhiều tài nguyên
      - Khởi động chậm hơn.
  - Architecture
    - Docker Client
      - Vai trò
        - Giao diện người dùng cho phép chúng ta tương tác với Docker Daemon thông qua các lệnh.
      - Cách hoạt động:
        - Khi chạy các lệnh như docker run, docker build, hoặc docker pull
        - Docker Client sẽ gửi yêu cầu này đến Docker Daemon thông qua REST API.
        - Docker Client có thể giao tiếp với Docker Daemon cục bộ hoặc từ xa.
      - Giao diện CLI
        - Người dùng sử dụng Docker CLI để thực thi lệnh.
        - Ví dụ: docker run nginx yêu cầu Docker Daemon tải và chạy container từ image nginx.
    - Docker Daemon (dockerd)
      - Vai trò
        - Docker Daemon chịu trách nhiệm chính trong việc xử lý các yêu cầu từ Docker Client
        - Quản lý Docker Objects
          - Containers
          - Images
          - Networks
          - Volumes
      - Cách hoạt động:
        - Quản lý vòng đời của container.
        - Xây dựng, lưu trữ và truy xuất các Docker Images.
        - Tương tác với hệ thống tệp và kernel Linux thông qua cgroups và namespaces.
      - Tính năng:
        - Isolation
          - Sử dụng công nghệ containerization để cô lập ứng dụng.
        - Orchestration
          - Tích hợp với các công cụ như Docker Swarm hoặc Kubernetes để quản lý container ở quy mô lớn.
    - Docker Registry
      - Vai trò
        - Lưu trữ
        - phân phối Docker Images.
      - Các loại Docker Registry:
        - Docker Hub
          - Docker Registry công cộng, miễn phí với các image phổ biến như nginx, redis, v.v.
        - Private Registry
          - Doanh nghiệp có thể tự triển khai registry riêng.
      - Cách hoạt động:
        - docker pull <image>
          - Docker Client sẽ tải image từ Docker Registry.
          - docker pull nginx
        - docker push <image>
          - Docker Client sẽ đẩy image lên registry.
          - docker push myrepo/myimage:latest
    - Nền tảng hỗ trợ Docker
      - Namespaces
        - Cô lập tài nguyên hệ thống (PID, mạng, hệ thống tệp, IPC).
      - Control Groups (cgroups)
        - Quản lý tài nguyên (CPU, bộ nhớ, I/O) mà container sử dụng.
      - Union File Systems
        - Hỗ trợ cấu trúc layers của Docker Images.
      - Container Runtime:
        - Trước đây: Docker sử dụng container runtime riêng.
        - Hiện nay: Docker sử dụng containerd và tuân theo tiêu chuẩn OCI (Open Container Initiative).
    - Docker Compose
      - Vai trò
        - Công cụ để định nghĩa và chạy nhiều container trong cùng một ứng dụng.
      - Cách hoạt động:
        - Định nghĩa cấu hình cho ứng dụng gồm nhiều container trong file docker-compose.yml.
        - Command: docker-compose up để khởi động tất cả container.
      - Commands
        - docker-compose up
          - Mục đích: Khởi động tất cả các dịch vụ được định nghĩa trong file docker-compose.yml.
          - Cách sử dụng:
            - Chạy lần đầu:
              - docker compose up
              - Tải xuống các images (nếu chưa tồn tại).
              - Tạo container và các network liên quan.
              - Kích hoạt các container.
            - Chạy nền (detached mode):
              - docker-compose up -d
              - Container sẽ chạy trong nền.
            - Force build lại:
              - docker-compose up --build
              - Xây dựng lại image trước khi khởi động.
        - docker-compose down
          - Mục đích: Tắt tất cả các container và dọn dẹp các tài nguyên liên quan (mạng, volumes, v.v.).
          - Cách sử dụng:
            - Lệnh cơ bản:
              - docker-compose down
              - Dừng container và xóa network.
            - Xóa cả volumes:
              - docker-compose down -v
              - Dọn dẹp volumes liên quan.
        - docker-compose ps
          - Mục đích: Liệt kê trạng thái của các container được quản lý bởi Compose.
          - Cách sử dụng:
            - Lệnh cơ bản:
              - docker-compose ps
              - Hiển thị danh sách các container, trạng thái và các cổng kết nối.
        - docker-compose build
          - Mục đích: Xây dựng hoặc rebuild các images từ file Dockerfile được định nghĩa trong docker-compose.yml.
          - Cách sử dụng:
            - Lệnh cơ bản:
              - docker-compose build
              - Xây dựng tất cả services.
            - Build một service cụ thể:
              - docker-compose build <service_name>
            - Không sử dụng cache:
              - docker-compose build --no-cache
        - docker-compose start
          - Mục đích: Khởi động lại các container đã dừng mà không tạo lại chúng.
          - Cách sử dụng:
            - Lệnh cơ bản:
              - docker-compose start
            - Start một service cụ thể:
              - docker-compose start <service_name>
        - docker-compose stop
          - Mục đích: Dừng các container mà không xóa chúng.
          - Cách sử dụng:
            - Lệnh cơ bản:
              - docker-compose stop
            - Stop một service cụ thể:
              - docker-compose stop <service_name>
        - docker-compose restart
          - Mục đích: Dừng và khởi động lại các container.
          - Cách sử dụng:
            - Lệnh cơ bản:
              - docker-compose restart
            - Restart một service cụ thể:
              - docker-compose restart <service_name>
        - docker-compose logs
          - Mục đích: Hiển thị log từ các container đang chạy.
          - Cách sử dụng:
            - Xem tất cả logs:
              - docker-compose logs
            - Xem log của một service cụ thể:
              - docker-compose logs <service_name>
            - Theo dõi log realtime:
              - docker-compose logs -f
        - docker-compose rm
          - Mục đích: Xóa container đã dừng.
          - Cách sử dụng:
            - Lệnh cơ bản:
              - docker-compose rm
            - Xóa container không cần xác nhận:
              - docker-compose rm -f
        - docker-compose config
          - Mục đích: Kiểm tra và xác thực file docker-compose.yml.
          - Cách sử dụng:
            - Lệnh cơ bản:
              - docker-compose config
            - Kiểm tra lỗi cú pháp và hiển thị cấu hình hợp lệ.
    - Docker Swarm
      - Vai trò
        - Công cụ tích hợp để quản lý và điều phối container trong môi trường phân tán.
      - Chức năng chính:
        - Tự động load balancing giữa các node.
          - Node là các máy tính chạy container
        - Dễ dàng mở rộng (scaling) số lượng container.
        - Quản lý các dịch vụ container trên nhiều host.
      - Command
        - docker swarm init để khởi tạo cluster.
        - docker service create --replicas 3 nginx để tạo 3 container nginx.
  - Core
    - dockerfile
      - What
        - Là một tệp văn bản đơn giản chứa các lệnh (instructions) để xây dựng một Docker Image.
        - Docker sử dụng tệp này làm đầu vào để tạo ra một Image, từ đó triển khai các container.
        - File
        - List Command/Instructions
        - To Create Image
      - Use
        - Cách thiết lập môi trường hệ điều hành.
        - Các thư viện, dependencies cần thiết.
        - Cách chạy ứng dụng hoặc dịch vụ.
      - Core Command
        - 1. FROM
          - Xác định image cơ sở (base image) để xây dựng.
          - Ví dụ:
            - FROM ubuntu:20.04
            - Image cơ sở là Ubuntu phiên bản 20.04.
        - 2. RUN
          - Chạy các lệnh trong quá trình build image.
          - Ví dụ:
            - RUN apt-get update && apt-get install -y python3
            - Cập nhật hệ thống và cài đặt Python3.
        - 3. COPY
          - Sao chép file/directory từ máy host vào image.
          - Ví dụ:
            - COPY app.py /app/
            - Sao chép file app.py vào thư mục /app/ trong image.
        - 4. ADD
          - Tương tự COPY, nhưng hỗ trợ giải nén tệp nén (e.g., .tar).
          - Ví dụ:
            - ADD archive.tar.gz /data/
        - 5. WORKDIR
          - Mục đích: Thiết lập thư mục làm việc mặc định cho các lệnh sau.
          - Ví dụ:
            - WORKDIR /app
        - 6. CMD
          - Xác định lệnh mặc định được chạy khi container khởi động.
          - Ví dụ:
            - CMD ["python3", "/app/app.py"]
            - Chạy file app.py bằng Python3 khi container bắt đầu.
        - 7. ENTRYPOINT
          - Tương tự CMD, nhưng không thể bị ghi đè bởi tham số khi chạy container.
          - Ví dụ:
            - ENTRYPOINT ["python3", "/app/app.py"]
        - 8. ENV
          - Thiết lập biến môi trường trong image.
          - Ví dụ:
            - ENV APP_ENV production
        - 9. EXPOSE
          - Mục đích: Khai báo cổng mà container sẽ lắng nghe.
          - Ví dụ:
            - EXPOSE 8080
        - 10. VOLUME
          - Mục đích: Tạo volume để lưu trữ dữ liệu bền vững.
          - Ví dụ:
            - VOLUME /data
        - 11. ARG
          - Định nghĩa các biến cho quá trình build (build-time variables).
          - Ví dụ:
            - ARG VERSION=1.0
        - 12. LABEL
          - Mục đích: Thêm metadata vào image.
          - Ví dụ:
            - LABEL maintainer="your-email@example.com"
    - image
      - What
        - Template
        - Immutable
          - Docker Images không thể bị thay đổi sau khi tạo.
        - Layered Architecture:
          - Image được xây dựng từ nhiều lớp (layers).
          - Mỗi lớp đại diện cho một thay đổi trong file hệ thống, giúp tối ưu hóa lưu trữ vì chỉ các lớp mới cần được tải xuống.
        - Create from Dockerfile
        - Create container.
      - include
        - Mã nguồn của ứng dụng.
        - Các thư viện và dependencies.
        - Cấu hình môi trường và hệ điều hành.
      - Example
        - nginx
        - redis
        - mysql
        - mssqlserver
        - jdk
      - Sử dụng
        - Pull image từ Docker Registry:
          - docker pull nginx
        - Liệt kê tất cả images:
          - docker images
        - Xóa image:
          - docker rmi my-image
    - container
      - What
        - Là các instance chạy thực tế của một Docker Image.
        - Là môi trường runtime cô lập, nhẹ, được thiết kế để chạy một ứng dụng hoặc dịch vụ cụ thể.
      - Features
        - Isolation
          - Chạy trong môi trường cô lập
          - Không ảnh hưởng đến các ứng dụng khác.
        - Ephemeral (dễ thay đổi):
          - Không lưu trữ dữ liệu bền vững (trừ khi sử dụng volumes).
        - Lightweight:
          - Container chia sẻ kernel với hệ điều hành chủ
          - Giảm overhead so với máy ảo.
      - How
        - Tạo và chạy container:
          - docker run -d --name my-container nginx
        - Dừng container:
          - docker stop my-container
        - Xóa container:
          - docker rm my-container
        - Xem log của container:
          - docker logs my-container
    - registry
      - Định nghĩa:
        - Là nơi lưu trữ các Docker Images.
      - Type
        - Public Registry:
          - Docker Hub: Registry công khai mặc định.
          - Ví dụ: docker pull nginx tải image từ Docker Hub.
        - Private Registry:
          - Doanh nghiệp có thể triển khai registry riêng để bảo mật.
      - How
        - Push image lên registry:
          - docker tag my-image my-registry.com/my-image:latest
          - docker push my-registry.com/my-image:latest
        - Pull image từ registry:
          - docker pull my-registry.com/my-image:latest
    - volumes
      - What
        - Volumes là cơ chế lưu trữ dữ liệu bền vững ngoài container.
        - Dữ liệu trong volumes tồn tại ngay cả khi container bị xóa hoặc khởi động lại.
      - Features
        - Quản lý bởi Docker, không phụ thuộc vào hệ thống file của host.
        - Có thể chia sẻ giữa nhiều container.
      - How
        - Tạo volume:
          - docker volume create my-volume
        - Gắn volume vào container:
          - docker run -d --name my-container -v my-volume:/data nginx
        - Liệt kê volumes:
          - docker volume ls
        - Xóa volume:
          - docker volume rm my-volume
    - networks
      - What
        - Cho phép các container giao tiếp với nhau hoặc với bên ngoài.
      - Type
        - Bridge (Mặc định):
          - Kết nối container trong cùng một host.
          - Ví dụ: docker network create my-bridge
        - Host:
          - Container chia sẻ mạng với host.
          - Ví dụ: docker run --network host my-container
        - Overlay:
          - Kết nối container trên nhiều host, dùng cho Docker Swarm.
        - None:
          - Không cấu hình mạng, container không thể giao tiếp với mạng.
          - Ví dụ: docker run --network none my-container
      - How
        - Tạo mạng:
          - docker network create my-network
        - Gắn container vào mạng:
          - docker run --name my-container --network my-network nginx
        - Liệt kê networks:
          - docker network ls
        - Xóa mạng:
          - docker network rm my-network
    - docker compose
      - Công cụ giúp quản lý nhiều container cùng một lúc
      - docker-compose.yml
      - Commands
        - docker-compose up
          - Khởi động tất cả các dịch vụ.
        - docker-compose down
          - Dừng và xóa tất cả các container.
  - Docker Process
    - Developer viết code và định nghĩa ứng dụng trong file Dockerfile.
    - Docker CLI
      - build image từ Dockerfile
      - push lên Docker Registry.
    - Docker Daemon tải image từ Docker Registry khi cần.
    - Docker Daemon khởi chạy container từ image.
    - Dữ liệu container được lưu trữ trong volumes (nếu cần).
    - Container giao tiếp với nhau hoặc với bên ngoài thông qua networks.
    - Ket hop nhieu container cung hoat dong
      - Docker Swarm
      - Kubernetes
  - Ưu/Nhược điểm
    - Ưu
      - Portability (Tính di động)
        - Chạy nhất quán trên mọi hệ điều hành.
      - Isolation (Cách ly)
        - Mỗi container hoạt động độc lập với các container khác.
      - Nhanh và nhẹ
        - Container khởi động nhanh và tiêu tốn ít tài nguyên.
      - Quản lý dễ dàng
        - Thông qua các lệnh và Docker Compose.
      - Khả năng mở rộng
        - Dễ dàng triển khai và mở rộng ứng dụng theo mô hình microservices
    - Nhược
      - Hiệu năng
        - Container không hoàn toàn cách ly như VM
        - Có thể ảnh hưởng đến hiệu năng.
      - Cần kiến thức nền tảng
        - Để tối ưu hóa, bạn cần hiểu rõ về hệ điều hành, mạng, và bảo mật.
      - Không phù hợp với mọi ứng dụng
        - Một số ứng dụng cần tính năng của VM hoặc phần cứng chuyên biệt.
  - Sử dụng
    - Development
      - Docker giúp tạo môi trường phát triển giống hệt với môi trường sản xuất.
    - CI/CD
      - Docker hỗ trợ các pipeline tự động hóa.
    - Microservices
      - Quản lý nhiều services nhỏ gọn và độc lập.
    - Học tập và thử nghiệm
      - Thiết lập nhanh chóng các môi trường khác nhau.
      - Tiết kiệm
- CICD
  - CI
    - What
      - Continous Integration
      - Tích hợp công cụ quản lý, đánh giá, vận hành mã nguồn tự động
      - Tự động hoá quy trình
    - Mục tiêu
      - Sớm phát hiện lỗi trong quá trình phát triển
      - Tăng cường hợp tác giữa các member
      - Tự động hoá quy trình đảm bảo, kiểm tra, kiểm thử code
      - Cải thiện hiệu suất
      - Code luôn hoạt động được ở 1 giai đoạn nhất định
    - Steps
      - Remote Repository
        - Mục đích
          - Lưu trữ code
          - Commit code
        - Tools
          - Github
          - AWS CodeCommit
          - Azure DevOps
          - Gitlab
          - Git Server
      - Xây dựng mã nguồn
        - Java
          - Maven
          - Gradle
        - Build
        - Compile
          - JDK
        - CLI
          - mvn test
          - mvn clean
          - mvn install
      - Đảm bảo cú pháp, format, convention
        - JS
          - ESlint
        - Python
          - Pylint
        - Java
          - SonarQueue
        - Other
          - SonarLint
      - Tự động chạy kiểm thử
        - Kiểm thử liên quan code
          - Unit Test
          - Integration Test
        - Analysis code
          - SonarQube
      - Tự động thông báo
        - CI server
        - Mục tiêu
          - Ghi logs hệ thống khi build CI
          - Thông báo lỗi khi build: thành công/thất bại
    - Process
      - Commit code
        - Dev commit code
        - Code đẩy lên git Remote
      - Trigger CI Pipeline
        - Dev commit and push code
        - Tự động kích hoạt Pipeline
        - Thực thi 1 loạt task vụ tự đông:
          - Kiểm tra code
            - Naming convention
              - Class
              - Variable
              - ....
              - Interface
              - Method
            - Biến khai báo ko được sử dụng tới
            - Spacing, new line
          - Kiểm thử
            - mvn test
            - types
              - integration tests
                - between multiple method/services/repositories
              - unit tests
                - method
          - Compile
            - Builde code
      - Feedback/Notification
        - Khi pipeline failed
          - Thông báo
        - Khi pipeline success
          - Thông báo
      - Fix/ Lặp đi lặp lại quá trình CI
    - Tools
      - CI Server
        - Github Actions
        - Azure DevOps
        - AWS CodeBuild/CodeCommit
        - TravisCI
        - Gitlab CICD
        - CircleCI
        - Jenkins
          - Tự cài đặt để sử dụng
      - Linting, Đảm bảo chất lượng mã
        - CheckStyle
          - Java
        - ESLint
        - SonarQube
      - Test Tools
        - Java
          - TestNG
          - JUnit
        - Angular
          - Karma
          - Jest
        - React
          - Jest
      - Development Tools
        - Java
          - Maven
          - Gradle
        - JS
          - Yarn
          - npm
          - vite
  - CD
    - What
      - Mở rộng từ CI
      - Hoặc chỉ cần thao tác thủ công nhỏ nhất có thể
      - Là 1 quy trình
      - Đảm bảo tích hợp quy trình triển khai sản phẩm tự động
    - Core Process
      - Continuous Integration (CI):
        - Code được hợp nhất (merge/pull request) vào nhánh chính (main/master branch).
        - Tự động unit test, integration test.
      - Build Artifact:
        - Tạo ra các bản build ổn định (artifact)
          - WAR, JAR (Java)
          - Docker image
      - Test Automation:
        - Các bài kiểm thử chi tiết như UI test, API test, hoặc performance test được thực hiện tự động để đảm bảo chất lượng sản phẩm.
      - Environment Provisioning:
        - Terraform, Ansible, hoặc Kubernetes để tự động hóa việc cấu hình môi trường triển khai.
      - Deployment Pipeline:
        - Thiết lập pipeline
        - Tự động hóa các giai đoạn kiểm tra và triển khai lên các môi trường
          - Development
          - Staging
          - Production
    - Lợi ích của Continuous Delivery
      - Rút ngắn thời gian phát hành sản phẩm:
        - Giảm thời gian từ khi mã được viết cho đến khi sản phẩm ra mắt.
      - Tăng cường sự hợp tác giữa các nhóm:
        - Các nhóm phát triển, kiểm thử và vận hành làm việc chặt chẽ để đảm bảo quy trình suôn sẻ.
      - Phát hiện lỗi sớm:
        - Quy trình kiểm thử liên tục giúp phát hiện lỗi trong giai đoạn phát triển thay vì sau khi triển khai.
      - Giảm rủi ro triển khai:
        - Triển khai thường xuyên, với từng thay đổi nhỏ, giúp dễ dàng xác định nguyên nhân lỗi nếu xảy ra.
      - Cải thiện tính linh hoạt:
        - Dễ dàng phản hồi với các yêu cầu thay đổi từ khách hàng hoặc thị trường.
    - Relationships
      - Continuous Integration
        - Tích hợp liên tục
        - Đảm bảo mọi thay đổi trong code đều được tích hợp vào repository chính
        - kiểm thử thường xuyên.
      - Continuous Deployment
        - Tiếp nối CI
        - Đảm bảo các thay đổi luôn sẵn sàng phát hành (Continuous Delivery)
        - Hoặc được triển khai trực tiếp (Continuous Deployment).
    - Compare
      - Tự động triển khai
        - Continuous Delivery
          - Chuẩn bị sẵn sàng, nhưng cần sự đồng ý (manual approval).
        - Continuous Deployment
          - Tự động triển khai trực tiếp đến Production.
      - Độ kiểm soát
        - Continuous Delivery
          - Nhiều kiểm soát hơn, phù hợp với các tổ chức lớn.
        - Continuous Deployment
          - Phù hợp với các tổ chức nhanh nhẹn (agile).
      - Ứng dụng chính
        - Continuous Delivery
          - Khi không phải mọi bản build đều cần phát hành ngay.
        - Continuous Deployment
          - Khi muốn cập nhật nhanh nhất đến người dùng.
    - Khó khăn khi CD
      - Thiếu quy trình tự động hóa:
        - Tự động hóa toàn bộ pipeline có thể phức tạp và cần thời gian.
      - Phụ thuộc vào các bài kiểm thử:
        - Chất lượng pipeline phụ thuộc vào việc viết và duy trì các bài kiểm thử tốt.
      - Đồng bộ giữa các nhóm:
        - Cần sự phối hợp chặt chẽ giữa nhóm phát triển, kiểm thử và vận hành.
    - Tools
      - Pipeline Tools
        - Jenkins
        - GitLab CI/CD
        - GitHub Actions
        - Azure DevOps
        - CircleCI
      - Containerization
        - Docker
        - Kubernetes
      - Configuration Management
        - Chef
        - Ansible
        - Puppet
      - Monitoring
        - Prometheus
        - Grafana
        - ELK Stack
- Docker Compose
  - What
    - docker-compose.yaml
    - Định nghĩa và quản lý các ứng dụng Docker nhiều container
  - Core
    - Image:
      - Image Docker nào sẽ được sử dụng cho service.
    - Ports:
      - Các cổng nào sẽ được ánh xạ giữa host và container.
    - Volumes:
      - Các volume nào sẽ được sử dụng để lưu trữ dữ liệu.
    - Dependencies:
      - Các service nào phụ thuộc vào service nào.
    - Environment variables:
      - Các biến môi trường nào sẽ được thiết lập cho container.
    - Networks:
      - Các mạng ảo nào sẽ được tạo ra để các container giao tiếp với nhau.
  - Commands
    - docker-compose up
      - Tạo và khởi chạy tất cả các services được định nghĩa trong file docker-compose.yml.
      - Thêm flag -d để chạy ở chế độ detached (nền).
    - docker-compose down
      - Dừng và xóa tất cả các container, networks và volumes được tạo bởi docker-compose up.
    - docker-compose start
      - Khởi động các services đã được tạo nhưng đang ở trạng thái dừng.
    - docker-compose stop
      - Dừng các services đang chạy.
    - docker-compose restart
      - Khởi động lại các services.
    - docker-compose ps:
      - Hiển thị trạng thái của các services.
    - docker-compose logs <service_name>
      - Xem logs của một service cụ thể.
    - docker-compose exec <service_name> <command>
      - Thực thi một command bên trong một container đang chạy.
      - Ví dụ: docker-compose exec web bash để truy cập vào shell của container web.
    - docker-compose build
      - Build lại các image cho các services (thường cần thiết khi bạn thay đổi Dockerfile).
    - docker-compose scale <service_name>=<number>
      - Thay đổi số lượng container đang chạy cho một service.
      - Ví dụ: docker-compose scale web=3 để chạy 3 container cho service web.
    - docker-compose config
      - Hiển thị cấu hình đã được hợp nhất từ file docker-compose.yml.
  - Process
    - Tạo Dockerfile cho từng service
      - Nếu bạn tự xây dựng image, hãy tạo một Dockerfile cho mỗi service trong ứng dụng của bạn.
    - Tạo file docker-compose.yml
      - Định nghĩa tất cả các services, networks, volumes và các cấu hình liên quan trong file này.
    - Chạy lệnh docker-compose up
      - Docker Compose sẽ tự động build (nếu cần) và khởi chạy tất cả các container theo cấu hình.
    - Truy cập và kiểm tra ứng dụng
      - Sau khi các container chạy thành công, bạn có thể truy cập ứng dụng thông qua các cổng đã được ánh xạ.
    - Thao tác Docker Compose
      - Để quản lý, xem logs, thực thi lệnh bên trong container, v.v.
      - Khi hoàn thành, chạy docker-compose down
        - Để dừng và xóa tất cả các tài nguyên đã được tạo.
  - Advanced
    - Environment Variables:
      - Sử dụng file .env để quản lý các biến môi trường chung cho toàn bộ ứng dụng.
    - Extending Services:
      - Định nghĩa các service chung và sau đó mở rộng chúng trong các service khác để tránh trùng lặp cấu hình.
    - Profiles:
      - Định nghĩa các profile để kích hoạt các nhóm services khác nhau tùy thuộc vào môi trường (ví dụ: development, test, production).
    - Health Checks:
      - Định nghĩa các health check cho các services để Docker Compose có thể tự động khởi động lại các container bị lỗi.